ISIM
Institut Supérieur d'Informatique de Médenine

Année Universitaire
2025-2026

Travaux Pratiques - Développement Web Avancé

TP N°2 : Routes et Contrôleurs

Classe : MP1ILC **Enseignant :** M. JARRAY Mondher **Durée :** 1h30 **Projet :** TaskFlow

Objectifs du TP

- Créer des routes avec différentes méthodes HTTP
- Utiliser des paramètres de route
- Créer et utiliser des contrôleurs
- Organiser les routes avec des groupes
- Mettre en place la structure de base de TaskFlow

Prérequis

- Avoir terminé le TP 1 (projet TaskFlow créé)
- Laravel 11 installé et fonctionnel
- Comprendre les bases du routage (Chapitre 2)

Partie 1 : Routes de Base (25 min)

Exercice 1.1 : Créer des routes simples

Ouvrez le fichier `routes/web.php` et ajoutez les routes suivantes :

- **a** Une route GET `/a-propos` qui retourne le texte "À propos de TaskFlow"
- **b** Une route GET `/contact` qui retourne le texte "Page de contact"
- **c** Une route GET `/services` qui retourne un tableau JSON avec 3 services

Aide

Pour retourner du JSON :

```php
Route::get('/services', function() {
    return response()->json([
        'service1' => 'Gestion de projets',
        'service2' => '...'
    ]);
});
```

Testez vos routes en visitant les URLs dans votre navigateur.

Écrivez votre code ici :

Exercice 1.2 : Routes avec paramètres

Créez les routes suivantes avec des paramètres :

a `/utilisateur/{nom}` qui affiche "Bienvenue [nom] !"

b `/projet/{id}` où l'id doit être un nombre
Utilisez `whereNumber()`

c `/equipe/{nom?}` avec un paramètre optionnel
Si pas de nom : "Équipe par défaut"

d `/projet/{projetId}/tache/{tacheId}` avec deux paramètres

```
Écrivez votre code ici :
```

Questions :

1. Que se passe-t-il si vous visitez `/projet/abc` (avec la contrainte whereNumber) ?

2. Quelle est la différence entre `{id}` et `{id?}` ?

Exercice 1.3 : Routes nommées

Ajoutez des noms à vos routes avec la méthode `name()` :

```php
// Exemple
Route::get('/a-propos', ...)->name('about');
Route::get('/contact', ...)->name('contact');
Route::get('/projet/{id}', ...)->name('projets.show');
```

Vérifiez vos routes avec la commande :

```
$ php artisan route:list
```

Capture d'écran : Résultat de la commande route:list

Partie 2 : Les Contrôleurs (35 min)

Exercice 2.1 : Créer le contrôleur ProjetController

Créez un contrôleur de ressource pour gérer les projets :

Étape 1

Exécutez la commande Artisan :

```
$ php artisan make:controller ProjetController --resource
```

Étape 2

Ouvrez le fichier créé dans `app/Http/Controllers/ProjetController.php`

Étape 3

Complétez les méthodes suivantes :

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class ProjetController extends Controller
{
    // Liste des projets
    public function index()
    {
        // Retourner un tableau de projets (simulé)
        $projets = [
            ['id' => 1, 'nom' => 'Site E-commerce', 'statut' => 'En cours'],
            ['id' => 2, 'nom' => 'Application Mobile', 'statut' => 'Terminé'],
            ['id' => 3, 'nom' => 'API REST', 'statut' => 'En attente'],
        ];

        return response()->json($projets);
    }

    // Afficher un projet spécifique
    public function show($id)
    {
        // À compléter : retourner les détails du projet $id
    }

    // Formulaire de création
    public function create()
    {
        return "Formulaire de création de projet";
    }

    // Enregistrer un nouveau projet
    public function store(Request $request)
    {
        // À compléter : afficher les données reçues
    }
}
```

Complétez les méthodes show() et store()

Code de la méthode show() :

Code de la méthode store() :

Exercice 2.2 : Lier les routes au contrôleur

Dans `routes/web.php`, remplacez vos anciennes routes par :

```php
use App\Http\Controllers\ProjetController;

// Route resource pour les projets
Route::resource('projets', ProjetController::class);
```

Exécutez `php artisan route:list` et répondez :

1. Combien de routes ont été créées automatiquement ?

2. Quelle est l'URL pour accéder à la méthode `edit` du projet 5 ?

3. Quelle méthode HTTP est utilisée pour la route `projets.destroy` ?

Exercice 2.3 : Créer le contrôleur TacheController

Créez un deuxième contrôleur pour gérer les tâches :

```
$ php artisan make:controller TacheController --resource
```

Implémentez les méthodes suivantes :

a. **index()** : retourne une liste de 5 tâches (simulées)

b. **show($id)** : retourne les détails d'une tâche

c. **store(Request $request)** : affiche les données reçues en JSON

Structure d'une tâche

```php
[
    'id' => 1,
    'titre' => 'Créer la base de données',
    'description' => 'Définir le schéma de la BDD',
    'statut' => 'todo', // todo, in_progress, done
    'priorite' => 'haute' // basse, moyenne, haute
]
```

Ajoutez la route resource dans web.php :

```php
Route::resource('taches', TacheController::class);
```

Partie 3 : Organisation Avancée (20 min)

Exercice 3.1 : Groupes de routes

Organisez vos routes avec des préfixes :

```php
// Routes publiques
Route::get('/', function() {
    return 'Bienvenue sur TaskFlow';
})->name('home');

Route::get('/a-propos', function() {
    return 'À propos de TaskFlow';
})->name('about');

// Routes de l'application (préfixe /app)
Route::prefix('app')->name('app.')->group(function() {
    Route::resource('projets', ProjetController::class);
    Route::resource('taches', TacheController::class);
});

// Routes API (préfixe /api)
Route::prefix('api')->name('api.')->group(function() {
    Route::get('/projets', [ProjetController::class, 'index'])
        ->name('projets.index');
    Route::get('/taches', [TacheController::class, 'index'])
        ->name('taches.index');
});
```

Après avoir ajouté ce code, répondez :

1. Quelle est l'URL complète pour créer un nouveau projet ?

2. Quel est le nom complet de la route pour lister les tâches dans l'application ?

Exercice 3.2 : Tâches imbriquées dans les projets

Les tâches appartiennent à un projet. Créez des routes imbriquées :

```php
// URL : /projets/{projet}/taches
Route::resource('projets.taches', TacheController::class);
```

Modifiez la méthode `index` de TacheController pour accepter l'ID du projet :

```php
public function index($projetId)
{
    // Simuler les tâches du projet
    $taches = [
        ['id' => 1, 'projet_id' => $projetId, 'titre' => 'Tâche 1'],
        ['id' => 2, 'projet_id' => $projetId, 'titre' => 'Tâche 2'],
    ];

    return response()->json([
        'projet_id' => $projetId,
        'taches' => $taches
    ]);
}
```

Testez l'URL `/projets/1/taches` et faites une capture :

Capture d'écran : Résultat de /projets/1/taches

Barème d'évaluation

Routes simples (1.1)	Routes avec paramètres (1.2)
Routes nommées (1.3)	ProjetController (2.1)
Routes resource (2.2)	TacheController (2.3)
Groupes de routes (3.1)	Routes imbriquées (3.2)

Total

☆ Exercice Bonus

1. Créez un contrôleur `DashboardController` avec une méthode `index` qui retourne un résumé :
   - Nombre total de projets
   - Nombre de tâches par statut
2. Ajoutez une route `/dashboard` qui utilise ce contrôleur
3. Créez un middleware fictif qui ajoute un header "X-TaskFlow-Version: 1.0" à toutes les réponses

Livrables à rendre

- 🗀 Fichier routes/web.php complet
- 🗀 Fichier ProjetController.php
- 🗀 Fichier TacheController.php
- 🗀 Captures d'écran des tests dans le navigateur
- 🗀 Capture de la commande php artisan route:list
- 🗀 Réponses aux questions